C Primer: Arrays, Pointers & Dynamic Memory

Study notes for Systems Programming

Covers every slide of **SPC5 (Arrays and Pointer Basics, 48 slides)** and **SPC6 (Dynamic Memory Allocation, 22 slides)**, including the speaker notes, the diagrams, and the practice questions, with worked answers.

How to read these notes

KEY IDEA
Ideas the slides stress and that you should be able to explain from memory.

WATCH OUT
Bugs, traps and exam-style gotchas.

BEYOND THE SLIDES
Extra explanation I added that is *not* on the slides (clarifications, why it works, safer idioms). Check your instructor's expectations before relying on these.

ANSWER
Worked answers to the questions the slides pose.

Each section header lists which slides it comes from, e.g. *SPC5 slides 3-4*. Memory addresses in examples assume a 4-byte `int` and 4-byte addresses, like the slides do.

Contents

Contents 2

Part 1: Arrays and Pointer Basics 3

1.1 Arrays: the basics . . . 3

1.2 Initializing character arrays and strings . . . 4

1.3 Arrays as automatic (stack) variables and VLAs . . . 4

1.4 Array assignment . . . 6

1.5 Arrays and functions . . . 6

1.6 Multidimensional arrays . . . 7

1.7 Pointers . . . 7

1.8 Revisiting the array-argument example . . . 9

1.9 Pointer declarations and pitfalls . . . 9

1.10 Strings in depth . . . 10

1.11 Pass by value vs. pass by reference (swap) . . . 12

1.12 Case study: scanf . . . 13

1.13 Practice questions with answers . . . 14

1.14 Part 1 at a glance . . . 14

Part 2: Dynamic Memory Allocation 15

2.1 The memory model: three pools . . . 15

2.2 Requesting memory: malloc . . . 16

2.3 calloc: allocate and zero . . . 17

2.4 realloc: adjusting the size . . . 17

2.5 Deallocation: free, and the two key rules . . . 18

2.6 Pointers and arrays . . . 18

2.7 Arrays of pointers and command-line arguments . . . 19

2.8 Allocating a 2D array dynamically . . . 20

2.9 Pointers taking the address of ...: lifetimes . . . 21

Part 3: Quick reference 22

3.1 Heap function cheat sheet . . . 22

3.2 Common bugs (from both decks) . . . 22

3.3 Self-test . . . 23

Part 1: Arrays and Pointer Basics

Source: SPC5, slides 1-48

1.1 Arrays: the basics

SPC5 slides 2-4

An **array** is a linear, **contiguous** collection of things, where every element has the **same fixed type**: an array of `char`, of `int`, of `double`, of booleans, even arrays of arrays. Like a `struct`, an array is a **type constructor**: it builds a new type out of an existing one.

- Declare with the size in brackets: `int x[5];` is five ints.
- Indexing starts at **0**, so the last valid index is `size - 1` (here 4). Access looks like a Python list: `x[0] = 1;`.
- `x[4] = x[3] + 1;` shows that elements are ordinary variables you can read and write.

Ways to initialize an array

```
int x[5];                      // define an array of 5 ints (NOT initialized)
x[0] = 1; x[1] = 2; x[2] = 3; x[3] = 4; x[4] = x[3] + 1;

int y[5] = {1, 2, 3, 4, 5};    // initialize with a list
int z[]  = {1, 2, 3, 4, 5};    // size optional if every element is listed (=5)
int a[5] = {1, 2};             // first 2 set; the rest become 0  -> 1,2,0,0,0
int b[5] = {1, 2, [4] = 5};    // C99 designated initializer      -> 1,2,0,0,5
```

Speaker-note details (C99 designated initializers). A designator `[k] = v` sets element `k`; values that follow it continue from index `k+1`.

```
int n[5] = {[4]=5, [0]=1, 2, 3, 4};   // holds 1,2,3,4,5

int a[MAX] = { 1, 3, 5, 7, 9, [MAX-5] = 8, 6, 4, 2, 0 };
// MAX = 6  : holds 1,8,6,4,2,0
// MAX = 13 : holds 1,3,5,7,9,0,0,0,8,6,4,2,0   (a "sparse array")
```

Also from the notes: standard C does **not** allow an empty initializer `{}`, but `gcc` and C++ do.

Arrays in memory

Elements sit side by side. For `int y[5] = {1,2,3,4,5};` with 4-byte ints starting at address 1000:

	Address	Value
y[0]	1000	1
y[1]	1004	2
y[2]	1008	3
y[3]	1012	4
y[4]	1016	5

> **KEY IDEA**
> Address of element i = **base address + i x sizeof(element)**. That one formula answers most 'what is the address of...' questions (see the practice questions in 1.12).

1.2 Initializing character arrays and strings

SPC5 slide 5

A **string** is a `char` array that **ends with a 0** (the null character, `'\0'`). That terminating 0 occupies a byte and **is part of the string**. You can initialize a string two ways: a list of characters or a double-quoted literal.

```
#include <stdio.h>
int main()
{
   char s[6] = {'H','e','l','l','o','\0'};
   char t[6] = "Hello";
   char u[]  = "Hello";          // compiler counts: 6 bytes
   printf("Array is: %s\n", s);
}
```

	Address	Value
s[0]	500	'H'
s[1]	501	'e'
s[2]	502	'l'
s[3]	503	'l'
s[4]	504	'o'
s[5]	505	0

> **WATCH OUT**
> "Hello" has 5 visible characters but needs **6 bytes**. If you declare `char t[5] = "Hello";` there is no room for the terminator, so it is no longer a valid string.

1.3 Arrays as automatic (stack) variables and VLAs

SPC5 slides 6, 27-30

You can declare arrays inside any function or block. They are **automatic** variables: created when you enter the function/block, **destroyed when you leave**. They live on the **stack** and are **not initialized** (they contain garbage until you write to them).

Variable length arrays (VLAs, C99): the size may depend on function arguments or other values known at that moment.

```
int foo(int n, int k) {
    int x[n];                 // n is known at this time
                              // like other auto variables: on the stack, NOT initialized
    for (int i = 0; i < n; i++)
        x[i] = 0;
    ......
    return -1;
}
```

Another example: 1D and 2D VLAs

```
void foo(int n)
{
    int i;
    int a[n];                   // array of n integers
    int b[2][n];                // 2-by-n array
    // indexes: a[0], a[1], ..., a[n-1].  NOT a[n]!
    for (i = 0; i < n; i++)     // typical loop of n times
        a[i] = i;
    // copy a into row 0 of b. You cannot write b[0] = a
    for (i = 0; i < n; i++)
        b[0][i] = a[i];
}
```

Speaker note: `n`'s value must be known when the declaration executes.

Questions about a local array (slide 27)

Given `int foo() { int x[5]; x[0]=1; ... x[4]=5; return 0; }`:

Question	Answer from the slide
Where is the array allocated?	It is automatic, so it is on the **stack**.
When is it allocated?	When you **enter the function**.
When is it deallocated?	When the function **returns**.
What about recursion?	**Each invocation gets its own copy.**
What if you access `x[5]`?	You are accessing **memory that is not yours** (out of bounds).
What if you access `x[-1]`?	Same as above.

> **BEYOND THE SLIDES**
> C does **no bounds checking**. `x[5]` or `x[-1]` may crash (segmentation fault), may silently read garbage, or may silently overwrite another variable. This is **undefined behavior**: the compiler and runtime promise nothing.

Automatic arrays: summary (slide 28)

- **Allocated** on entering the function, **deallocated** on leaving it (both automatic).
- **NOT initialized**.
- Exist **directly on the stack** like other local variables.
- Size can be a **static constant**, or a **dynamic expression** (C99).
- **Cannot be too big**, because the stack is limited.

VLA support in C (slide 29)

- VLAs were **introduced in C99** and are **optional in C11** (a compiler may not support them).
- In some applications you do not want large amounts of data on the stack.
- If VLAs are unsupported (or the data is big), you must keep the array on the **heap**. You will learn how in Part 2.

1.4 Array assignment

SPC5 slide 7

You **cannot assign a whole array** to another array, even when the types match.

```
int main() {
   int x[10];
   int y[20];
   int z[10];
   x = y;       // error
   x = z;       // error, even though both are int[10]
}
// a.c: error: assignment to expression with array type
```

To copy an array you must copy **element by element** (a loop), exactly as the slides do with `b[0][i] = a[i]`. The same applies to strings (see `strcpy`, section 1.10).

> **KEY IDEA**
> Speaker note: in **Java**, arrays are heap allocated. In **C**, arrays can be **stack allocated**.

1.5 Arrays and functions

SPC5 slides 8-9

The calling convention in C is **by value for everything, except arrays**. Arrays are always passed **by reference**: the function receives the array's address (a "pointer", introduced in 1.6). Also:

- Modifications a function makes to an array argument **are visible to the caller**.
- **Functions cannot return arrays** (there are no easy array assignments).

```
int foo(int x[], int k) {
    for (int i = 0; i < k; i++)
        x[i] = i;
    return x[0];
}

int main() {
    int t[5];
    int a0 = foo(t, 5);
    int a1 = t[4];
    printf("%d %d\n", a0, a1);     // prints: 0 4
}
```

The slide's stack picture (stack grows downward in the diagram; each function has its own **frame**):

foo's frame	i
	x (address of t)
	k (5)
main's frame	a1
	a0
	t[0] ... t[4] (the 5-int array)

KEY IDEA
`x` inside `foo` does **not** hold five ints. It holds **the address of** `t` (one pointer-sized value). So `x[i] = i` writes directly into `main`'s array. That is why `main` sees the change.

BEYOND THE SLIDES
Because only an address is passed, `foo` has **no way to know the array's length**. That is why the length `k` is passed as a separate argument. Writing `int x[]` or `int *x` as a parameter means exactly the same thing (see 1.7).

1.6 Multidimensional arrays

SPC5 slide 10 (and slide 30)

```
int h[2][3] = { {0, 1, 2}, {10, 11, 12} };
```

Logically it is a table (rows by columns). In memory it is a single contiguous strip stored **row by row**: all of row 0 first, then row 1; within each row, column 0 first, then column 1, and so on. This is **row-major order**.

	0	1	2
0	0	1	2
1	10	11	12

	Address	Value
`h[0][0]`	1000	0
`h[0][1]`	1004	1
`h[0][2]`	1008	2
`h[1][0]`	1012	10
`h[1][1]`	1016	11
`h[1][2]`	1020	12

KEY IDEA
Address of `h[i][j]` = **base + (i x number_of_columns + j) x sizeof(element)**. For `h`: `h[1][2]` is at 1000 + (1x3 + 2)x4 = **1020**.

You **cannot** write `b[0] = a` to copy a row (that is an array assignment); loop over `j` instead.

1.7 Pointers

SPC5 slides 11-19

Pointers are "perhaps the scariest part of C" and also "the most useful part of C". A pointer is simply **a value denoting the address of a memory cell**.

Variables and memory (slide 12)

- Memory is an **array of bytes**. Every byte is numbered; that number is its **address**.
- An address is just an **unsigned integer**.
- Every variable lives in memory and is associated with **two numbers**: its **address** and the **value stored at that address**.

Implicit address use (slide 13)

The compiler uses addresses all the time, even when you never write one:

```
int foo (int v)
{
    int a, b;      // allocate storage space for a and b
    a = 1;         // store 1 to memory, at a's address
    b = a;         // load value from a's address, write it to b's address
    v = v + b;     // load v, load b, add, write result to v's address
    return v;
}
```

	Address	Value
a	100004	1
b	100008	
(next)	100012	

Explicit use: pointers (slide 14)

A **pointer is a variable that holds the address of something**.

```
int * p;      // declare p to be a pointer to an int
```

- The **value of** `p` is the address of an `int`.
- `p` **itself has an address** (it is a variable stored in memory too).
- Speaker note, what can you do with addresses? (1) **Get** the address of something, (2) **dereference** an address to get the value stored there, (3) **compute** the address of something (pointer arithmetic, next lecture).

The two operators: & and * (slide 15)

Operator	Name	Meaning
&	Reference	"get" the address of something
*	Dereference	"use" the address (go to the memory it names)

```
int x = 10, y;
int *px;       // px is a pointer to int; px itself has an address

px = &x;       // &x is the address of x. Save it in px
y  = *px;      // use px as an address: read the value there, save in y
*px = 20;      // write 20 at the location px points to -> x becomes 20
```

Picture it: tracing the code (slides 16-19)

Assume 32-bit ints and addresses. `x` is at 1020, `y` at 1016, `px` at 1012. Each box below is the memory after the highlighted statement runs.

After the declarations

	Address	Value
x	1020	10
y	1016	?
px	1012	?

After px = &x;

	Address	Value
x	1020	10
y	1016	?
px	1012	1020

After y = *px;

	Address	Value
x	1020	10
y	1016	10
px	1012	1020

After *px = 20;

	Address	Value
x	1020	20
y	1016	10
px	1012	1020

KEY IDEA

`px = &x` stores the **address** 1020 into `px`. `*px` then means "the int at address 1020" which **is** `x`. So `*px = 20` changes `x` itself, and `y` (a copy made earlier) stays 10.

WATCH OUT

`*` means two different things: in a **declaration** (`int *px;`) it says "px is a pointer"; in an **expression** (`y = *px;`) it means "dereference".

1.8 Revisiting the array-argument example

SPC5 slide 20

```
int foo(int x[], int k) {              // array syntax
    for (int i = 0; i < k; i++)
        x[i] = i;
    return x[0];
}

int foo(int * x, int k) {              // x is the address of an int
    for (int i = 0; i < k; i++)
        x[i] = i;      // use the pointer as an array
    return x[0];
}
```

The two versions are the same. `x` **is the starting address of an array, which is the same as the address of element 0.** That explains why arrays are "passed by reference": the thing copied into the parameter is the address, and `x[i]` works on any pointer.

1.9 Pointer declarations and pitfalls

SPC5 slides 21, 31-32

```
int*    p;
int  *  p;
int    *p;
```

All three declare `p` **to be a pointer to an integer**. The first makes that statement clear; the second is "non-committing"; the third says that **what** `p` **points to is an integer** (classic C style).

Pitfall: the * binds to the variable, not the type

```
int  *a, b;
int* c, d;
int  e, *f;
int  *g, *h;
```

Type int * (pointer)	Type int (plain integer)
a, c, f, g, h	b, d, e

WATCH OUT
`int* c, d;` looks like two pointers but **only `c` is one**; `d` is a plain `int`. To declare two pointers you need a `*` on each name, as in `int *g, *h;`.

"Answer this!" (slide 21)

The slide poses two questions. Here are worked answers:

ANSWER
Why do pointers have a type (`int *p` vs `char *p`) if they only store an address? Because the address alone does not say what is stored there. The type tells the compiler (1) **how many bytes** to read or write when you dereference (1 for `char`, 4 for `int`, ...), (2) **how to interpret** those bytes, and (3) how far `p + 1` moves (it jumps by `sizeof` the pointed-to type: pointer arithmetic, next lecture).

ANSWER
Does a pointer have an address, and how do you store it? Yes: a pointer is a variable, so it lives in memory and has an address (`&p`). To store that address you need a **pointer to a pointer**, e.g. `int **pp = &p;`. You will use exactly this type for `int **rows` in Part 2.

1.10 Strings in depth

SPC5 slides 33-38

In C, strings are **just `char` arrays with a NUL (`'\0'`) terminator**. There are two kinds you will meet:

- A **string literal** such as `"a cat"`: the compiler allocates memory to hold every character you can see **plus the terminating `'\0'`**. A literal **cannot be changed by the program** (a very common bug).
- **All other strings** must have space allocated for them by the program (a `char` array or heap memory).

"a cat" in memory: `a`, (space), `c`, `a`, `t`, `\0`, six bytes. Speaker note: the string's value, when you use it in an expression, is the **address of its first character**.

String literals cannot be changed (slide 34)

```
#include <ctype.h>
char *makeBig(char *s) {
  s[0] = toupper(s[0]);      // tries to modify the string
  return s;
}
int main()
{
  makeBig("a cat");          // modifies a literal!
}
// $ ./string
// Segmentation fault (core dumped)
```

BEYOND THE SLIDES
Why the crash: literals are constants stored in the **static/global pool** (SPC6 slide 3), which the OS marks read-only. A fix is to copy the text into your own writable array first: `char buf[] = "a cat"; makeBig(buf);`.

Walking a string (slide 35)

```
char line[100];
...
for (i = 0; i < 100 && line[i]; i++) {   // in range, and not at the '\0' yet
  if (isalpha(line[i]))
     putchar(line[i]);                   // print one character
}
```

The loop condition has two checks: the **index must be in range**, and `line[i]` must be non-zero (**not the end of the string yet**), because `'\0'` is 0 and 0 is false.

Referring to a string through a pointer (slide 36)

```
// str is the address of a char, namely 'm', the first char of the string
// str[0] is 'm', str[1] is 'y', and so on
char *str = "my string";

char *s;       // s is a pointer to char
s = str;       // s now refers to the SAME string as str
```

Speaker notes: you can walk a string with a pointer: `for (s = str; *s; s++) putchar(*s);` (and `s = str` means `s = &str[0]`).

Copying strings (slide 37)

```
char buf[100];               // buf is a char array
char *str = "a cat";         // str refers to the string literal
strcpy(buf, str);            // copies the characters AND the '\0'   (string.h)
char *s = buf;               // s refers to the string inside buf, different from str
```

> **KEY IDEA**
> **Copying content vs. copying a pointer to content.** `s = t;` copies only the pointer: afterwards `s` and `t` refer to the **same memory**. To get an independent copy of the characters, use `strcpy` (or a loop). And `char mybuffer[100]; mybuffer = "a cat";` **does not compile**: that is an array assignment.

A beginner's strcpy (slide 38)

Idea: copy one character at a time until the `'\0'` has been copied.

```
char * strcpy(char *dest, char *src)
{
    unsigned int i = 0;
    // copy any character that is not '\0'
    for (i = 0; src[i] != '\0'; i++)
        dest[i] = src[i];
    dest[i] = src[i];        // copy the '\0' too
    return dest;
}
```

> **BEYOND THE SLIDES**
> The loop stops **before** copying the terminator, so the line after the loop copies it explicitly. Without that, `dest` would not be a valid string. Note also that `strcpy` does **not** check that `dest` is big enough; it needs at least `strlen(src) + 1` bytes or you overflow the buffer. (The library version takes `const char *src`.)

1.11 Pass by value vs. pass by reference (swap)

SPC5 slides 39-40

Goal: write a function that swaps two `int` variables. Because parameters are passed **by value**, this version does **not** work:

```
void swap(int i, int j)
{
    int k = j;
    j = i;
    i = k;
}
int main()
{
    int a=1, b=2;
    swap(a, b);           // a and b are unchanged: still 1 and 2
    return 0;
}
```

`swap` gets its own copies `i` and `j` in **its own stack frame** (alongside the local `k`). It swaps the copies, then the frame disappears. `main`'s `a` and `b` are never touched.

The fix: pass **pointers** to the variables (simulating pass-by-reference):

```
void swap(int *i, int *j)
{
    int k = *j;
    *j = *i;
    *i = k;
}
int main()
{
    int a=1, b=2;
    swap(&a, &b);         // pass the ADDRESSES
    return 0;
}
```

Now `i` and `j` hold the addresses of `a` and `b`; `*i` and `*j` read and write `main`'s variables directly.

> **KEY IDEA**
> Rule of thumb: **to let a function modify a variable of the caller, pass its address (`&var`) and dereference it (`*p`) in the function.** Arrays already arrive as addresses, so they need no `&`.

1.12 Case study: scanf

SPC5 slides 41-48

```
#include <stdio.h>
int main()
{
    char name[128];
    int pears  = 0;
    int apples = 0;
    scanf("%s %d %d", name, &pears, &apples);
    printf("%s ate %d apples and %d pears.\n", name, apples, pears);
    return 0;
}
```

What is going on? We pass `scanf` **values** (the format string's address, plus three more), and `scanf` must **write results into our variables**. So it needs their addresses:

Argument	What is passed	Why
`"%s %d %d"`	address of the format string (a literal)	tells scanf what to read
`name`	address of the 128-byte array (already an address)	no `&` needed: an array already is a pointer, so scanf **can write there too**
`&pears`	address of the int	`&` needed, otherwise scanf would only get a copy of 0
`&apples`	address of the int	`&` needed, same reason

The slides (42-48) build up the picture step by step: `main`**'s frame** holds `name` (128 bytes), `pears` and `apples`; calling `scanf` creates `scanf`**'s frame** with its argument slots (the slides label them a1, a2, a3 next to the `format` pointer) which receive the format string and the addresses. Then `scanf` follows those addresses and writes the parsed values into `main`'s variables.

> **WATCH OUT**
> Forgetting `&` on `pears` or `apples` is a classic bug: `scanf` would treat the **value** (0) as an address and write to the wrong place (typically crashing). Do **not** put `&` on `name`, because it is an array.

> **BEYOND THE SLIDES**
> `%s` without a width has no length limit. Typing more than 127 characters overflows `name[128]`. A safer format is `%127s`.

1.13 Practice questions with answers

SPC5 slides 22-25, 27

Slide	Question	Answer
22	`int a[] = {1,2,3,4}; int b[4] = {1,2,3};` How many integers in `a` and in `b`?	**4** and **4**. `a`'s size is counted from the initializer list; `b` is declared with 4, and its missing last element is set to 0 (b = 1,2,3,0).
23	`char c[] = {'a','b','c','d'}; char d[] = "abcd";` How many bytes in `c`? in `d`? What is `sizeof(d)`?	`c` has **4** bytes (no terminator, so it is **not** a string). `d` has **5** bytes (4 letters + `'\0'`). `sizeof(d)` = **5**.
24	`int a[10];` If `a[0]` is at 1000, where is `a[4]`?	1000 + 4 x 4 = **1016**.
25	`char t[10][20];` If `t[0][0]` is at 1000, where is `t[1][1]`?	Each row has 20 chars (1 byte each): 1000 + 1x20 + 1 = **1021**.

Slide 26 says "Study the remaining slides yourself". Those slides (27-48) are all covered above: automatic arrays (1.3), strings (1.10), swap (1.11), and scanf (1.12).

1.14 Part 1 at a glance

Topic	Remember
Array	Contiguous, same-type elements; indices 0..n-1; no bounds checking.
Local array	Stack, automatic lifetime, not initialized; VLA size can be a runtime value (C99).
Array assignment	`a = b` does not compile; copy element by element.
Array parameter	Passed as an address; callee can modify caller's data; pass the length separately; cannot return arrays.
2D array layout	Row-major; `h[i][j]` at base + (i*cols + j)*sizeof.
Pointer	A variable holding an address; `&` gets an address, `*` follows it.
Pointer type	Tells how many bytes to access and how to interpret them.
`int *a, b;`	`a` is a pointer, `b` is an int.
String	`char` array ending with `'\0'`; literals are read-only; use `strcpy` to copy content.
Modify caller's variable	Pass `&var`; dereference with `*` in the callee (swap, scanf).

Part 2: Dynamic Memory Allocation

Source: SPC6, slides 1-22

2.1 The memory model: three pools

SPC6 slides 2-5

A program's memory is divided into **three pools**. Each has a **different lifetime** and a **different allocation/deallocation policy**. The slide's diagram runs from address `0x00000000` (bottom) to `0xffffffff` (top): the **stack** is near the top and grows **downward**, the **heap** grows **upward** from the lower region, with free space between them, and static/global data sits at the bottom.

	Static / global	Stack	Heap
What lives here	All **constants** (including string literals), **global variables**, all variables declared `static`	**Local variables** in functions	Memory from manual, on-the-fly allocations (`malloc` etc.)
Allocated	When the program **starts**	**Automatically** when entering the function	When **you** call the allocator
Deallocated	When the program **terminates**	**Automatically** when leaving the function	When **you** call `free`
Who is in charge	Compiler / loader	Compiler (automatic)	**The programmer**
Size	**Fixed**: compiler must know the size and reserve it	Limited (default with gcc: **2 MB** per the slide)	Grows as you request
Lifetime	Whole program	Scope of the function (do not use after it returns, e.g. via a pointer)	As long as the block is **not freed**

KEY IDEA

Stack memory is **easy to manage because it is automatic**, but it should **not be used after the function returns**, for example indirectly through a pointer. If you need to enlarge it for **large arrays or deep recursion**, you have to increase the stack size.

BEYOND THE SLIDES

The 2 MB figure is the slide's claim; the real limit depends on the OS and settings (on Linux, `ulimit -s` shows it).

2.2 Requesting memory: malloc

SPC6 slides 6-9

```
#include <stdlib.h>
void* malloc(size_t size);
```

- `size_t` is an **unsigned integer type** (the slide places it in `<stdlib.h>`) used to represent **sizes of objects in bytes**.
- If successful, `malloc(n)` returns a **generic pointer (`void *`)** to a block of `n` **bytes on the heap**.
- If it fails, it returns `NULL`.

```
char* p = malloc(100);     // request 100 bytes
```

Generic pointers: void* (slide 7)

A `void *` points to a memory block whose content is **un-typed**. It is used for **raw memory operations** and **generic functions**.

- **Automatic conversion** when assigned to another pointer type: `int * pox = malloc(6 * sizeof(int));` (no cast needed).
- **Requires a cast before dereferencing** (read/write), since the compiler does not know what is there: `*(int *)pv;` uses `pv` as an `int *`.
- `NULL` is a special pointer value used for initialization and error handling: `#define NULL ((void*)0)`.

Can't always get what you want (slide 8)

`malloc` **may fail**, for instance when you are out of memory. You then get `NULL`. There is not much to do except **report the error and terminate nicely**. The standard idiom:

```
char* p = malloc(100);          // request 100 bytes
if (p == NULL) {
    // report error and finish
    perror("Not enough memory");
    exit(1);
}
```

How much space? (slide 9)

You tell `malloc` how many **bytes** you need. For an array that is **number of elements x space per element**. `sizeof(T)` gives the number of bytes for a value of type `T`.

```
int* pox = malloc(6 * sizeof(int));   // space for 6 ints
if (pox == NULL)
    report error and finish;
```

> **WATCH OUT**
> Never write `malloc(6)` when you mean six ints; that requests 6 **bytes**. Always multiply by `sizeof` of the element type. Also, **always check for `NULL`** before using the pointer.

2.3 calloc: allocate and zero

SPC6 slide 10

```
#include <stdlib.h>
void* calloc(size_t nmemb, size_t size);

int* pox = calloc(6, sizeof(int));    // space for 6 ints, all set to 0
if (pox == NULL)
    report error and finish;
```

- `calloc` takes the **number of elements** and the **size of each** as separate arguments.
- The slide says it is implemented in terms of `malloc`; the key difference is that `calloc` **also initializes the content to 0**.
- `malloc`'d memory, by contrast, is **not** initialized.

2.4 realloc: adjusting the size

SPC6 slide 11

```
#include <stdlib.h>
void* realloc(void* ptr, size_t size);

char* p = malloc(100);      // request 100 bytes
...
p = realloc(p, 200);        // p may change!
```

What if you change your mind? You asked for 100 bytes but now need 200. Before calling `realloc(p, size)`, `p` must be either:

- a pointer returned by a **previous** `malloc`/`calloc`/`realloc`, or
- `NULL`, in which case the call is equivalent to `malloc(size)`.

> **WATCH OUT**
> The block may be **moved**, so `p` **may change**. Always use the pointer `realloc` returns, and do not keep using old copies of the old address.

> **BEYOND THE SLIDES**
> On success the old contents are preserved (up to the smaller of the old and new sizes). If `realloc` **fails** it returns `NULL` and the old block is still allocated, so `p = realloc(p, n)` would lose your only pointer and leak it. Safer: `tmp = realloc(p, n); if (tmp == NULL) {handle error} else p = tmp;`

2.5 Deallocation: free, and the two key rules

SPC6 slides 12-13

```
#include <stdlib.h>
void free(void *ptr);

free(ptr);
```

- Straightforward: call the library function `free` with a pointer to the block to free.
- **Do not free a pointer twice!**
- **After freeing a pointer, set it to `NULL`!**

The two key rules (slide 13)

Rule 1	**Everything you requested should be freed, eventually.**
Rule 2	**Only free what was allocated via `malloc`/`calloc`/`realloc`.**

Consequences of breaking the rules:

Violation	Consequence
Not freeing (Rule 1)	Memory **leaks**; your program will eventually run out of memory.
Freeing memory that was never allocated, or already freed (Rule 2)	**Undefined behavior** and horrible crashes. It may cause an immediate memory error and crash; worse, it may **corrupt the heap** and crash **later**; even worse, the program may **keep running, corrupting your data and writing it to disk without you realising**.

> **BEYOND THE SLIDES**
> Setting a freed pointer to `NULL` helps because `free(NULL)` is defined to do nothing, so an accidental second `free` becomes harmless, and a stale dereference crashes immediately instead of silently reading freed memory. Also never free pointers to stack or static variables (e.g. `free(&x)` or `free("literal")`): that breaks Rule 2.

2.6 Pointers and arrays

SPC6 slides 14-15

After `pox = malloc(6 * sizeof(int));` you have a block of **6 integers** (6 x 4 = 24 bytes) laid out back to back on the heap, with `pox` pointing to the first byte. **That looks like an array, and you can use `pox` as if it were an array.**

```
void doSomething(int n) {
  int * pox;
  pox = malloc(sizeof(int)*n);   // request memory from the heap
  *pox = 0;      // set the int at the address pox to 0
  pox[0] = 0;    // same thing
  pox[1] = 1;    // the int after pox[0]
  // more lines here ...
  free(pox);     // remember to free!
}
```

KEY IDEA

`*pox` and `pox[0]` are the same thing; `pox[i]` is the i-th int after the start. That is why the SPC5 example worked with `int *x` as if it were an array. The difference from a local array: heap memory's size can be decided at run time, it survives until you `free` it, and it is not limited by the stack size.

2.7 Arrays of pointers and command-line arguments

SPC6 slides 16-17

Pointers, like integers, can be placed in an array.

```
int a0;  int a1;  int a2;               // three separate ints ...
int a[3];                               // ... or an array of ints

char *p0 = malloc(10);                  // three separate pointers ...
char *p1 = malloc(10);
char *p2 = malloc(10);
char * p[3];                            // ... or an array of pointers

p[0] = malloc(10);                      // can also do it with a loop
p[1] = malloc(10);
p[2] = malloc(10);
```

Example: main's command-line arguments (slide 17)

```
int main (int argc, char *argv[]);
```

- `argc`: the **number of arguments** on the command line.
- `argv`: an **array of pointers to characters**. The number of elements is `argc`, and each element points to a **null-terminated string**.

For the command `$ gcc -o ab ab.c`:

Element	Points to	Note
`argv[0]`	`"gcc"`	the program name
`argv[1]`	`"-o"`	
`argv[2]`	`"ab"`	
`argv[3]`	`"ab.c"`	argc is 4 here

ANSWER

What is `argv[3][1]`? `argv[3]` is the string "ab.c"; index 1 of that string is `'b'`.

2.8 Allocating a 2D array dynamically

SPC6 slides 18-20

Static 2D arrays like `int h[2][3]` need constant sizes (or VLAs). With the heap you can pick the dimensions `m` x `n` at run time. The slides show **three approaches**.

Take 1: an array of row pointers

```
void doSomething(int m, int n)
{ int **rows;
  rows = malloc(sizeof(int *) * m);      // array of m pointers
  for (int i = 0; i < m; i++)
    rows[i] = malloc(sizeof(int)*n);     // one int array for each row
  ...
  for (int i = 0; i < m; i++)
    free(rows[i]);                       // free every row ...
  free(rows);                            // ... then the array of pointers
}
```

`rows` points to an array of `m` pointers (`rows[0]` ... `rows[m-1]`); each `rows[i]` points to its own separately allocated row of `n` ints. The slide asks: **"Looks similar to `argv`?"** Yes: `argv` is exactly an array of pointers to separately stored strings, and `rows` has the type `int **`, a pointer to a pointer.

> **WATCH OUT**
> Free in the right order: free each `rows[i]` **first**, then `rows`. If you free `rows` first, you lose the only copies of the row addresses and **leak** them. This approach makes **m + 1** `malloc` calls and needs **m + 1** `free` calls (Rule 1).

Take 2: one malloc for all the data

- Request **all the memory the data needs in a single `malloc()` call**, instead of calling `malloc` `m` times, once per row.
- Then **calculate** `rows[1]`, `rows[2]`, ... yourself so they point into the block (this uses **pointer arithmetic**, which is the next lecture).
- You can even calculate the address of any element directly (e.g. `rows[10][5]`).

The diagram: the `rows` array of pointers on one side, all pointing into **one contiguous strip** laid out as row 0, row 1, ..., row m-1.

> **BEYOND THE SLIDES**
> A sketch of the idea (my illustration, not on the slide; `data + i*n` is pointer arithmetic):
> `int *data = malloc(m * n * sizeof(int)); int **rows = malloc(m * sizeof(int *));`
> `for (int i = 0; i < m; i++) rows[i] = data + i * n;`
> Then `rows[i][j]` works as before, and you free with `free(rows); free(data);`.

Take 3: a pointer to an array of n ints

```
void doSomething(int m, int n)
{ int (* arr)[n];      // arr is a pointer to an array of n ints
  arr = malloc(m * n * sizeof(int));     // ONE malloc() for the data
  // to access elements
  arr[1][2] = 1;  arr[2][3] = arr[1][2] + 10;
  ...
  free(arr);           // ONE free
}
```

`arr` is a pointer to **an array of `n` ints**, so `arr[i]` is a whole row and `arr[i][j]` is one element. The compiler does the row arithmetic for you. The slide warns: `n` **must be a constant if VLAs are not supported.**

> **WATCH OUT**
> In `int (* arr)[n]` the **parentheses matter**. `int *arr[n]` would instead be an *array of n pointers* (the "array of pointers" from 2.7).

Comparing the three

	Take 1	Take 2	Take 3
Declaration	`int **rows`	`int **rows` (+ data block)	`int (*arr)[n]`
mallocs / frees	m + 1	1 for data (+1 for the row-pointer array)	1
Rows contiguous?	Not necessarily	Yes (data in one block)	Yes
Needs VLA/constant n?	No	No	Yes (n must be constant if VLA unsupported)
Access	`rows[i][j]`	`rows[i][j]`	`arr[i][j]`

Slide 21 says "Study the remaining slides yourself"; the one remaining slide is covered next.

2.9 Pointers taking the address of ...: lifetimes

SPC6 slide 22 (slide shows a skull-and-crossbones warning)

Whether a pointer is safe to use depends on **what it points to** and how long that thing lives:

Pointer to the address of a ...	Valid for how long?
static (or global)	Forever: the address never goes bad, since a static lives as long as the program.
stack (automatic) variable	Only **as long as the variable is**. When the function returns, the address is **bogus**.
heap block	As long as the block is: it disappears when **explicitly de-allocated (freed)**.

```
int *bad(void) {
    int local = 42;
    return &local;      // DANGER: local dies when bad() returns
}                       // the caller receives a "dangling pointer"
```

WATCH OUT
That skull-and-crossbones is about **dangling pointers**: pointers into memory that has gone away (a returned stack variable, or a freed heap block). Using them is undefined behavior: it may crash, or appear to work today and fail tomorrow. Return a heap pointer (or have the caller supply a buffer) instead.

Part 3: Quick reference

3.1 Heap function cheat sheet

Function	Signature	What it does
malloc	`void* malloc(size_t size)`	Allocate `size` bytes, **uninitialized**. Returns NULL on failure.
calloc	`void* calloc(size_t nmemb, size_t size)`	Allocate `nmemb` x `size` bytes, **zeroed**.
realloc	`void* realloc(void* ptr, size_t size)`	Resize a block (`ptr` from malloc/calloc/realloc, or NULL = malloc). Block **may move**.
free	`void free(void *ptr)`	Release a heap block. Never twice; set pointer to NULL after.

3.2 Common bugs (from both decks)

Bug	Symptom / why	Fix
Out-of-bounds index (`x[5]`, `x[-1]`)	Touches memory that is not yours; undefined behavior	Keep `0 <= i < n`
Assigning arrays (`x = y`)	Compile error	Loop, or `strcpy` for strings
Modifying a string literal	Segmentation fault	Copy into a `char` array first
`int* c, d;`	`d` is not a pointer	`int *c, *d;`
Missing `&` in `scanf`	Writes through a bogus address	`&var` for non-arrays
Forgetting the `'\0'` slot	String functions run off the end	Allocate length + 1
Not checking `malloc`'s result	NULL dereference	`if (p == NULL) ...`
Wrong `malloc` size	Heap overflow	`n * sizeof(T)`
Memory leak	Program eventually runs out of memory	Every allocation gets a `free` (Rule 1)
Double free / free non-heap	Crash now or heap corruption later	Free once; set to NULL; only free heap blocks (Rule 2)
Returning address of a local	Dangling pointer	Return heap memory or use caller's buffer
Freeing 2D rows in wrong order	Leaks the rows	Free `rows[i]` first, then `rows`

3.3 Self-test

Cover the answers and try these before an exam.

#	Question	Answer
1	Why can't you assign `x = y` for two `int[10]` arrays?	An array is not an assignable value; copy element by element.
2	What does `foo(t, 5)` actually pass when `t` is an `int[5]`?	The address of `t[0]` (arrays are passed by reference).
3	Where is `h[1][2]` in `int h[2][3]` at base 1000?	1000 + (1x3+2)x4 = 1020.
4	What do `&` and `*` do?	`&` gets an address; `*` dereferences (uses) an address.
5	What are the types in `int *a, b;`?	`a` is `int *`, `b` is `int`.
6	How many bytes does `char d[] = "abcd";` use?	5 (includes `'\0'`).
7	Why does `makeBig("a cat")` segfault?	It writes to a string literal, which is read-only.
8	Why does `swap(int i, int j)` fail?	Parameters are copies; the originals are untouched. Pass pointers.
9	Which pool holds: a global, a local, a `malloc` block, a string literal?	Static/global; stack; heap; static/global.
10	What does `malloc` return on failure, and what should you do?	`NULL`; report an error and exit (or handle it).
11	`malloc` vs `calloc`?	`calloc` zero-initializes (and takes count + size).
12	Why must you use the return value of `realloc`?	The block may move to a new address.
13	What are the two key rules of the heap?	Free everything you allocated; only free what you allocated.
14	What is `argv[3][1]` for `gcc -o ab ab.c`?	`'b'`.
15	How do you free a take-1 2D array?	Free each `rows[i]`, then `rows`.